
UniRL: Trajectory-Centric Post-Training Across Heterogeneous Generative Models

UniRL Authors

Affiliations to be added before the arXiv release

authors@to.be.added

Abstract

Reinforcement-learning systems share an outer loop—generate, score, assign credit, and update—yet their implementations fragment because autoregressive tokens, diffusion trajectories, multimodal values, and multi-stage rollouts carry different state and impose different execution constraints. We present UniRL, a trajectory-centric system that places a lineage-preserving, modality-aware intermediate representation at the boundary between post-training semantics and distributed execution. A trajectory batch is a sequence of typed frontiers encoding prompt-rooted trees, with decoded values, model conditions, replay segments, rewards, and behavior-policy versions. Rollout engines implement one transformation, `Sample` $\rightarrow$ `Sample`, while logical roles, placement, tensor transport, weight publication, and phase residency determine how the transformation executes. These contracts factor model-specific replay from lineage, movement, placement, and scheduling decisions; they do not assert that the underlying objectives or every engine–topology combination are interchangeable. The design is grounded in a source audit of UniRL commit `f5d7104`; subsequent experiment reports provide initial GPU evidence. Three-seed diffusion results show improved held-out automatic scores together with reduced within-prompt diversity, while an eight-GPU native-recipe comparison reports lower iteration time than VerL-Omni. A source-importing CPU harness exercises ten core IR invariants. The present evaluation distinguishes these reported measurements from independently regenerated raw evidence: the primary AR endpoint, real-rollout lineage validation, paired representation cost, and a controlled time-to-quality result remain open.

1 Introduction

Online post-training is a feedback program. It expands inputs into policy samples, evaluates those samples, turns feedback into a learning signal, and applies one or more optimizer updates. At this level, language-model GRPO, diffusion-model policy gradients, prompt-rewrite pipelines, and tool-using agents look deceptively similar. Their data paths do not. An autoregressive update may need packed token IDs, masks, rollout log probabilities, and per-token values; a diffusion update may need a sparse set of latent states, a noise schedule, and transition statistics; a composed workflow must retain which image descended from which rewritten prompt; an asynchronous run must additionally retain which policy version generated every sample.

Existing systems have made major progress on scalable LLM post-training and flexible device mapping (Sheng et al., 2024; Hu et al., 2024; Wang et al., 2025; NVIDIA Corporation, 2025). Fully or partially asynchronous systems address rollout tail latency and accelerator idle time (Fu et al., 2025; Zhang et al., 2026; Zhu et al., 2025). Meanwhile, diffusion RL methods and multimodal frameworks broaden the optimization and model families that can be trained (Black et al., 2023; Liu et al., 2025; Xue et al., 2025; Zheng et al., 2025; Huang et al., 2026). The remaining design question is how the information shared by these workflows should survive changes in their model-specific state and execution paths.

Can one post-training program cross generative model families and execution topologies while retaining the structure needed for correct grouping, replay, credit assignment, transport, and policy-version control?

Its answer is to make a trajectory intermediate representation (IR) the stable semantic object, then expose execution choices as explicit contracts around that object. The IR makes a complete prompt-rooted lineage, rather than an individual row or an LLM-specific tensor dictionary, the unit of scheduling and sharding. Typed leaves distinguish decoded multimodal values from model conditions and optimization segments. Distributed methods declare how each structured field scatters, concatenates, packs, or remains shared. Dense tensors can be replaced by row-addressable references without changing the trajectory structure. This representation is passed through a small set of logical roles and can be mapped to colocated or disjoint GPU slabs.

The paper develops two coupled design ideas:

- A *lineage-preserving trajectory IR* that separates decoded values, model conditions, and replay segments while retaining ancestry across generative stages (Section 3). Synthetic equality checks provide initial semantic evidence; recorded-rollout validation and the primary AR endpoint remain open alongside the reported diffusion results.
- A *factorization of semantic roles from physical execution*: rollout remains **Sample** $\rightarrow$ **Sample**, while field-aware dispatch, tensor references, placement, weight publication, and residency specify its execution (Section 4). Native-recipe and internal topology comparisons provide initial performance observations; the paired representation-cost control remains unmeasured.

Version-aware bounded staleness is one realization of the execution contracts (Section 4.5); an incomplete pilot is reported separately, without a controlled operating-region conclusion. The evidence therefore proceeds from structural correctness, through credible learning and a composed workflow, to measured system cost. The experiment IDs and corresponding artifacts are specified in Section 6.

The current evidence comprises an archived CPU regression and reported GPU results for diffusion training, native-recipe systems performance, and deployment topology. The diffusion result includes a material diversity regression; the systems comparison retains estimator and trainable-parameter differences. These observations do not establish an equivalent-algorithm speedup or complete the trajectory-centric claim: real-rollout semantic validation and a paired measurement of representation cost remain necessary.

2 Problem: Preserve Semantics While Changing Execution

Let an online post-training iteration consume a batch of roots X , sample trajectories with behavior policy π_v , obtain reward r , construct credit A , and update parameters θ :

$$X \xrightarrow{\pi_v} \mathcal{T} \xrightarrow{R} (\mathcal{T}, r) \xrightarrow{C} (\mathcal{T}, A) \xrightarrow{U_\theta} \theta'. \quad (1)$$

The compact notation hides three forms of heterogeneity.

Trajectory heterogeneity. The sufficient statistics for an update depend on the generative process. For AR policies, a segment may be a variable-length packed token sequence with masks, old log probabilities, values, and returns. For stochastic diffusion or flow policies, a segment may contain selected latent snapshots, a shared noise schedule, the indices of sampled transitions, and rollout or replay transition statistics. Multi-stage and agentic paths add nonuniform lineage and observations.

For example, let prompt p produce rewrites $w_{p,j}$ and images $x_{p,j,k}$. The image model conditions on $w_{p,j}$, but credit may compare either images under that rewrite or all images under p . A flattened image batch can implement both choices only if it retains or reconstructs the relevant parent keys. This example motivates explicit ancestry, while the image model’s replay state motivates typed segments; neither requirement follows from the other.

Role heterogeneity. Generation, reward evaluation, model replay, gradient computation, and policy publication have different dependencies and may use distinct runtime systems. Serving engines such as vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024) are optimized for generation, while FSDP (Zhao

et al., 2023) or another training backend owns parameters and optimizer state. Moving a large policy or trajectory between them is a first-order cost.

Topology heterogeneity. A small run may time-share one GPU set between training and rollout; a larger run may separate them; a variable-duration rollout may benefit from asynchronous admission. Each choice changes memory pressure, transfer paths, synchronization frequency, and allowable policy lag.

These observations produce four requirements.

Table 1: Design requirements and the corresponding UniRL mechanism. The right column states what must be evaluated; it is not a result.

Requirement	Mechanism	Evidence required
Preserve structure	Parent-addressed parts; root-complete split, select, concat, and reward propagation	Trace invariants across AR, diffusion, and cross-stage runs
Preserve model state	Typed primitives, conditions, and AR/latent replay segments	Rollout-replay consistency on exercised model paths
Change execution safely	Logical roles, placement scopes, field-aware dispatch, and tensor references	Matched-work topology and transport ablations
Control policy age	Version-stamped generated parts, lag filters, quiescent publication, and hard boundaries	Lag distributions, throughput, and time-to-quality

Non-goals. UniRL does not erase model-specific code: a model bundle still exposes architecture-specific stages, and an algorithm still defines its replay and loss. It does not automatically discover an optimal placement. The present runtime uses a driver to sequence roles and Ray actors (Moritz et al., 2018) to host them; it is not a new cluster scheduler. Finally, source-code breadth is not evidence that all combinations are production-ready.

3 A Trajectory-Centric Programming Model

3.1 A trajectory batch is a sequence of tree frontiers

The central type is

$$\mathcal{T} = [P_0, P_1, \dots, P_L], \quad P_i = (I_i, X_i, C_i, Z_i, R_i, A_i, V_i, M_i), \quad (2)$$

where I_i are sample identifiers, X_i decoded modality values, C_i model conditions, Z_i optimization segments, R_i rewards, A_i advantages or returns, V_i a behavior-policy version, and M_i auxiliary metadata. In code, $\mathcal{T}$ is a `Sample` and each P_i is a batched `Part`, rather than a single tree node. The sequence encodes a forest of root-prompt trees whose edges connect adjacent frontiers.

Identifiers encode ancestry. Root IDs contain no lineage separator; a branch appends a child ordinal. For every non-root row j in P_i , its parent ID must occur in P_{i-1} :

$$\forall i > 0, \forall s \in I_i : \text{parent}(s) \in I_{i-1}. \quad (3)$$

The constructor checks Equation 3. The `fork` operator appends k generated children per frontier row; `observe` appends a branch-one input from an environment; and `propagate_rewards` reduces child reward to an unscored parent.

Batch operations act on complete root groups. `Sample.split()` returns one tree-complete trajectory per root, and slicing or selection is implemented by selecting these groups rather than independent rows. Moving complete roots preserves the parent-contiguous order constructed by `fork`; independently permuting child rows need not preserve the credit computation.

The contract has preconditions. The constructor checks parent existence, not every property required by later operations. At the audited commit, group advantages require uniform, contiguous groups; reward

propagation reshapes child rewards into parent-major blocks and does not join them by ID. Position-wise `Sample.concat` also assumes equal frontier depth, and generic shared-field concatenation retains the first value without checking equality. Callers must therefore establish compatible schemas, group order, and shared values before merging; the async batch assembler separately rejects mixed behavior versions. E0 checks these conditions at the actual experiment boundaries. The representation supports the contract, but its constructor alone is not a general proof of correctness for arbitrary trees or permutations.

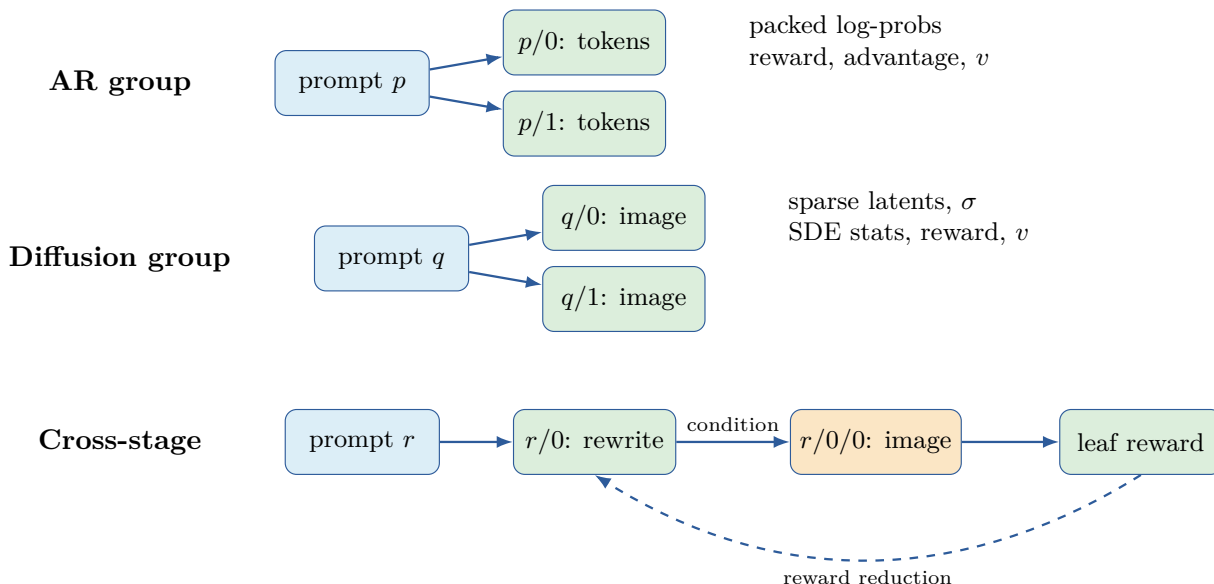


Figure 1: Three trajectory shapes represented by the same ordered-part IR. A generated part retains decoded values and the optimization segment required to replay its policy. The policy version v is attached to generated frontiers. The cross-stage path illustrates available ancestry; E4 freezes the rewrite model and trains only diffusion.

3.2 Values, conditions, and segments are separate layers

UNIRL distinguishes three kinds of leaves that are often conflated.

Primitives. Primitives are decoded or externally supplied values: text, images, videos, audio waveforms, and media references. Image pixels and variable-duration video/audio values use packed representations with per-example shapes or cumulative offsets. They are suitable for reward functions, visualization, and subsequent turns, but need not be the tensors replayed by the trainable model.

Conditions. Conditions are model-facing encodings, such as text embeddings or image latents. They allow rollout adapters and train-side stages to agree on typed inputs without prescribing one tokenizer, text encoder, or multimodal fusion architecture.

Segments. Segments retain optimization-facing state. `TextSegment` stores variable-length packed token fields. `LatentSegment` stores latent snapshots at selected indices, a shared sigma schedule, and optional SDE log probabilities or transition means. The sparse index map is important: a diffusion algorithm can replay only selected transitions instead of materializing an entire denoising path for every loss.

The distinction makes decoded payload lifetime independent of replay lifetime. After reward logging, trainers may drop large decoded media while retaining conditions, segments, reward, and credit for training.

3.3 A field algebra defines structural batching

Every structured batch field declares a composition rule: concatenated, packed variable-length, shared, or reduced by maximum, minimum, sum, or mean. These annotations drive concat, split, selection, repetition, and device movement recursively. They also define how a distributed method reconstructs its return value. For example, a DP-scattered rollout receives a tree-complete shard and returns parts that are merged position-wise; a broadcast lifecycle method returns one replicated control result.

This is a stronger contract than requiring each role to hand-author tensor slicing. It also creates a measurable cost: recursive structural operations and reference bookkeeping must be benchmarked in Section 6.3 rather than assumed free.

3.4 Rollout engines implement one transformation

Every rollout engine fills a request and returns a trajectory:

$$F_{\theta,e} : \mathcal{T}_{\text{request}} \longrightarrow \mathcal{T}_{\text{filled}}, \quad (4)$$

where e identifies a train-side or external generation runtime. The base contract also exposes lifecycle and policy-publication operations, but it does not standardize the engine’s internal serving API.

The current source contains four useful forms of Equation 4. The train-side adapter invokes the model pipeline directly under evaluation mode. Dedicated SGLang and vLLM-Omni adapters translate typed trajectory inputs to their native requests and rebuild typed responses. A composed engine serially fills multiple generated parts, for example an AR rewrite followed by diffusion generation. An agentic engine repeatedly appends policy turns and environment observations. The claim is therefore about a common boundary, not one universal generation kernel.

3.5 The algorithm boundary preserves specialized replay

A train stack owns microbatch planning, gradient accumulation, global loss normalization, gradient clipping, optimizer steps, and per-update aggregation. An algorithm object owns the stage-specific map

$$(C, Z, A; \theta) \longmapsto \mathcal{L} \quad \text{and backward.} \quad (5)$$

Before repeated optimizer updates, the stack can freeze an old-policy anchor at the same microbatch geometry used for training. AR objectives replay token log probabilities; diffusion objectives replay selected transitions and may compare rollout statistics, a pre-update replay, or a reference model. Supervised and forward-process objectives explicitly opt out of advantages.

This division is deliberately not advertised as algorithmic novelty. GRPO follows the group-relative formulation introduced for language reasoning (Shao et al., 2024); PPO follows proximal policy optimization (Schulman et al., 2017); diffusion objectives build on prior diffusion-RL methods (Black et al., 2023; Liu et al., 2025; Xue et al., 2025; Zheng et al., 2025). The systems question is whether these objectives can consume the same trajectory and execution contracts without hiding their different sufficient statistics.

4 From a Program to a Distributed Execution

Figure 2 separates the feedback program from the mechanisms that execute it.

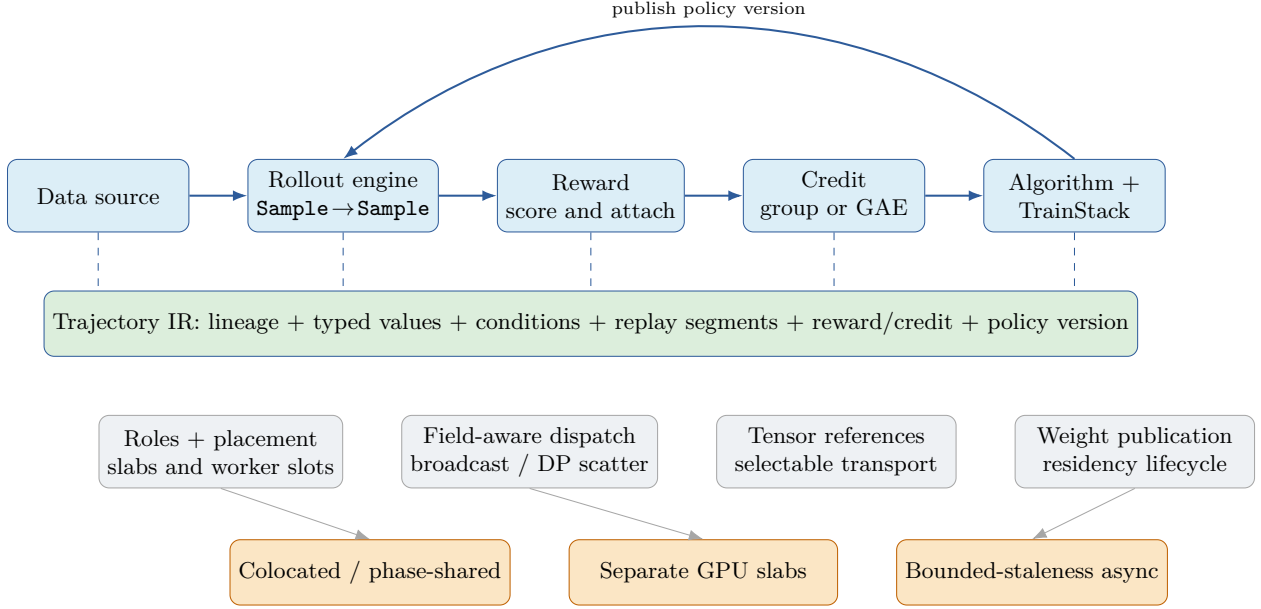


Figure 2: Source-derived UniRL architecture. The top row is the feedback program; the green bar is its stable data boundary. The lower rows map logical roles to communication and resource policies. Not every topology is valid for every engine; startup validation rejects unsupported combinations.

4.1 Logical roles, device slabs, and worker slots

A `DevicePool` creates one Ray worker per GPU initially and reserves one strict-packed placement group per node. A placement scope selects a fraction of currently available devices. Roles created inside a scope either share its worker slot—enabling local sibling access and memory time-sharing—or receive isolated slots on the same devices. Disjoint top-level scopes form separate training, rollout, or reward slabs.

A role is a regular Python object derived from `Remote`. The handle constructs one role replica on each selected worker and derives data-, tensor-, pipeline-, sequence-, and expert-parallel rank metadata from the role configuration. Method decorators state argument dispatch and return collection. The trainer is consequently a single-controller program, while each method may invoke multi-controller distributed training or serving inside a role. This resembles HybridFlow’s separation between orchestration and distributed computation (Sheng et al., 2024); UniRL’s paper claim centers instead on carrying the same typed lineage through heterogeneous generative stages.

4.2 Tensor references decouple structure from movement

Before an RPC, dense tensor leaves may be *dehydrated* into `TensorRefs`. A reference is an ordered list of half-open row spans over opaque handles, plus shape, dtype, and device metadata. Selection and contiguous slicing construct new span views without moving tensor data. At a destination, hydration resolves all referenced spans in a batch and reconstructs the original nested object.

The transport interface has a deliberately small contract: store tensors, resolve handles, identify references, and optionally localize references for destination workers. The audited source provides (i) a colocated per-worker store, (ii) a per-GPU store with IPC and NCCL transfers, and (iii) a `TransferQueue` path backed by simple storage or Mooncake-compatible storage. Mooncake itself was proposed for disaggregated serving (Qin et al., 2024); here its storage interface is a transport option rather than a claimed UniRL invention.

The driver can therefore manipulate trajectory structure and metadata without materializing all GPU tensors. Whether this reduces memory and transfer cost in practice is RQ3, not an assumption.

4.3 Phase lifecycles and policy publication

Training and generation often hold different copies or formats of a policy. UniRL exposes their interaction as an ordered lifecycle:

1. wake the rollout engine sufficiently to receive weights;
2. publish parameters if the cadence requires it;
3. optionally offload colocated training state;
4. generate and stamp the frontier with the engine’s policy version;
5. sleep the engine when the selected residency policy requires it;
6. restore training state before replay and optimization.

The concrete order varies by engine capabilities. For example, staged wake can receive weights before mapping the rest of an AR engine, and a train-side engine reads live model objects without a separate publication role. Separate slabs use an explicit one-time handshake. Weight-sync implementations cover LoRA tensor paths and several full-weight paths, including tensor serialization, CUDA IPC, NCCL, and checkpoint rendezvous. The base engine can also receive per-track updates for composed models.

The source rejects combinations whose lifecycle has not been validated, such as a train-side rollout on a physically separate slab or selected EMA/offload combinations. This fail-closed policy is important because a partially awakened engine or silently stale adapter can produce plausible but invalid training data. Section 6 therefore includes replay-consistency and failure-injection checks alongside throughput.

4.4 Synchronous execution

The synchronous AR trainer performs rollout, reward, credit assignment, replay/update, optional evaluation, and checkpointing in order. The diffusion trainer can accumulate multiple rollouts into one optimizer window and can place reward on the train slab or a third slab. In both cases, topology affects resource ownership and residency but not the trajectory boundary. This makes a controlled topology experiment possible: hold model, prompts, group size, reward, number of replayed steps or tokens, and optimizer updates constant while changing only engine or placement.

4.5 Bounded-staleness execution

Asynchrony changes correctness conditions, not just control flow. UniRL’s asynchronous AR and diffusion trainers share one manager. A background dispatcher fills every engine slot up to a configured per-worker capacity. Completed trajectories are grouped by root; only complete groups enter the ready buffer. Every generated part must carry an output policy version v_o .

Let v_t be the current optimizer-update version and u the number of optimizer updates performed per consumed rollout batch. The manager exposes lag in update units,

$$\ell = v_t - v_o \leq ub, \tag{6}$$

where b is the configured staleness allowance in rollout-batch units; the filter rejects work beyond that budget. Before publishing version v_t , it pauses admission, drains or suspends work, completes any started or batch-aligning prefix, checks that no publication splits a training batch across behavior versions, publishes weights, updates the rollout version, and then resubmits eligible pristine carry. Evaluation and checkpointing require a fully empty manager.

This discipline differs from claiming fully asynchronous optimization. Weight publication remains a barrier, and the current asynchronous diffusion path deliberately limits in-flight batch structure more tightly than the AR path. The evaluation must report the actual lag histogram, dropped work, and barrier time; a nominal “async” label is insufficient.

5 Instantiations of the Common Contracts

The following paths show which information crosses the common boundary and which computations remain specialized.

5.1 Autoregressive reasoning

An input part carries prompt text and metadata. Forking by group size creates generated shells with sibling IDs. An AR engine fills packed token segments and decoded text, a reward role attaches scalar or component rewards, and the part computes group-normalized credit or token-level generalized advantage estimates. The train stack replays the same segment and applies the selected objective. Optional token-length balancing permutes complete samples before DP sharding, preserving sibling identifiers and precomputed credit.

5.2 Diffusion and flow generation

A diffusion request forks prompts and records sampling parameters, a sigma schedule, selected SDE indices, and an initial-noise recipe. The engine returns decoded media plus a latent segment whose sparse trajectory fields are sufficient for the selected update. FlowGRPO-style objectives replay recorded steps and compare new transition log probabilities against a frozen rollout or replay anchor (Liu et al., 2025). Forward-process objectives such as DiffusionNFT use clean generated samples rather than the same reverse-trajectory likelihood path (Zheng et al., 2025). The common train stack handles microbatching and optimizer state; the algorithm remains responsible for these incompatible estimators.

5.3 Cross-stage and environment-interacting trajectories

In a prompt-enhancement path, one root produces one or more AR rewrites, and every rewrite produces one or more images. The image reward is attached to the leaf; when the rewrite stage trains, parent-major reduction supplies its feedback. The primary E4 case freezes that stage and uses ancestry to select the diffusion advantage groups. In a tool-interacting path, the policy emits an action-like text part, the environment appends an observation part, and the process repeats until termination or suspension. Both workflows need ancestry to survive rollout scheduling and sharding; flattening them into unrelated rows would require rebuilding that correspondence out of band.

E4 checks these groups against an explicit ID-based reference and then studies the two grouping objectives in training.

6 Evaluation: Reported Results and Remaining Evidence

We organize evaluation around four research questions. Each reported run must archive the resolved configuration, commit and dirty diff, model and dataset revisions, environment manifest, raw logs, structured metrics, parser version, and evaluation outputs. Primary comparisons use the same hardware one framework at a time. The operational commands, run-directory schema, and failure-routing rules are frozen in GPU_HANDOFF_README.md; the stricter evidence-admission contract is in EXPERIMENT_RUNBOOK.md.

Evidence scope and date. The online experiment report, retrieved on 2026-09-27, states an as-of date of 2026-09-22. We combine it with the result manuscript in pull request 2 at commit `ecce5097`; source snapshots, transcribed data, and a deterministic figure/table generator accompany this repository under `artifacts/reported_results/2026-09-22/`. The tables below are regenerated from those *reported summaries*, not from newly inspected training logs. The underlying per-run repositories were unavailable during this integration, so source identities, complete timing distributions, and all admission gates have not been independently revalidated here. We retain that distinction, avoid reconstructed learning curves or uncertainty bars, and list unresolved source discrepancies in `EXPERIMENT_RESULTS.md`. The settings below describe the prespecified protocol unless explicitly identified as an observed result; matching each setting to a resolved run configuration remains an artifact-release obligation. In contrast to the earlier report-derived

summaries, the HunyuanImage-3.0 and MiniMax-H3 reproduction bundles are generated directly from their per-step W&B histories and resolved training recipes.

Table 2: Experiment-to-claim map and reported status as of 2026-09-22. P0 rows remain necessary for the primary trajectory-centric claim.

ID	Priority	Controlled study	Paper evidence
E0	P0 gate	Correctness, configuration, and instrumentation checks	Correctness closure reported; evidence packaging remains incomplete
E1	P0	Qwen3-4B-Base on DAPO-Math	GRPO and DRPO each trained at one seed; primary external endpoint missing
E2	P0	SD3.5-Medium diffusion training	Three-seed UniRL scores and diversity reported; reference trained, not evaluated
E3	P0	Native UniRL-VeRL-Omni recipes, eight GPUs	Aggregate timings reported; strict semantic parity not established
E4	P0	Root versus rewrite grouping	Synthetic equality checks pass; no consistent three-seed quality advantage
E5	P0 minimum	Representation cost and deployment topology	Topology measured; paired representation cost not measured
E6	P1	Synchronous/asynchronous Qwen3 study	Single-seed pilots only; formal sweep canceled

The order is E0, then E1–E4 and the minimum E5 topology/cost slice; E6 and broader E5 scaling/transport backends follow. If compute is limited, we complete primary E0–E5 before adding more models, algorithms, or scaling points: without E4 the paper would not directly test lineage, and without the minimum E5 slice it would not bound the cost of that abstraction.

Environment and source identity. E3 and E5-T report a fixed total allocation of eight GPUs; the complete hardware, topology, environment, source-patch, model, data, and scorer manifests still need to accompany the released results. The older E1 incident report describes a particular 32-GPU allocation, but its hardware must not be silently reused to fill the E2/E3 manifests. Likewise, the design-audit commit is not a substitute for the later experiment commits.

6.1 RQ1: Does UniRL learn on representative workloads?

The first question is scientific rather than purely operational. A complete training-validity claim requires improvement over the frozen base together with satisfactory collapse diagnostics. E2 reports an improvement in automatic scores but triggers the diversity guard, so its result is a qualified quality–diversity trade-off. Reproducing another implementation additionally requires a full-budget, mathematically aligned reference; a 30-step speed run cannot supply that evidence. The E2-R reference is trained but has not been evaluated head-to-head, and the adapted AR reference remains absent.

AR workload. E1 trains Qwen3-4B-Base (Yang et al., 2025) with full-parameter GRPO on DAPO-Math-17k (Yu et al., 2025). The data name does not imply use of the DAPO optimizer; we fix the population-standard-deviation group estimator implemented in the audited recipe. Each rollout contains 64 prompts with eight samples (512 trajectories), uses SGLang TP=1 with thinking enabled, temperature/top- p 1.0, top- k disabled, and at most 8192 new tokens. The learner uses group-standard-deviation normalization, symmetric clipping at 0.2, `seq-mean-token-mean` aggregation, four updates per rollout, a 10,240-token microplanner, AdamW learning rate 10^{-6} and weight decay 0.01, and full-weight publication every rollout. We prespecify 800 rollouts on 32 total GPUs, internal AIME evaluation every ten rollouts, full checkpoints every 200 rollouts, and seed IDs 11, 22, and 33 applied to data order and the SGLang server RNG. The planned external evaluation covers the frozen base and checkpoints 200/400/600/800 on MATH-500 (500 problems, avg@4) and AIME 2024/2025 (30 problems each, avg@16), using the registry’s fixed decoding and `math_verify` grader. The UniRL run and the selected current reference must align prompts, chat template, verifier, maximum generation length, group size, effective batch, clipping, KL treatment, optimizer updates, and evaluation decoding. The public veRL Qwen3 FSDP example is only a scaffold because its stock model variant, data, group size, response budget, and KL/objective settings differ; the AR reference row remains blank until a pinned adapted launcher passes a resolved-config parity audit. The final 800-rollout checkpoint is primary. Internal evaluation uses AIME 2024/2025, so an AIME-selected best checkpoint is

reported only as validation-selected and cannot establish an untouched AIME test result. MATH-500 is the primary external AR endpoint, with a training/evaluation overlap audit required before interpretation; the fixed-budget AIME curves are secondary diagnostics. The protocol calls for every seed, final-checkpoint curves, and 95% confidence intervals; $\text{avg}@k$ denotes mean accuracy across k sampled answers, not $\text{pass}@k$.

E1 status: training complete, primary endpoint open. The current report records two full-budget single-seed arms: GRPO and an additional DRPO arm. Internal AIME changes from 0.0208 to 0.0177 for GRPO and from 0.0250 to 0.159 for DRPO. Both improve and then decline during training; these are validation observations under a different decoding budget from the primary MATH-500 endpoint. The external endpoint and across-seed replication remain missing, so the observations do not establish objective superiority or general AR training validity. The additional DRPO recipe and selection history also need to be documented alongside the original GRPO protocol. Issue 1 records a historical evaluation hang; the later report attributes its resolution to the bounded grader-process cleanup in upstream pull request 462 and records completed training. The older issue is therefore not evidence that E1 is still blocked before rollout zero.

Diffusion workload. E2 fixes SD3.5-Medium (Stability AI, 2024) and the UniRL FlowGRPO setting used by E3: 48 prompts with 16 images (768 samples) per rollout, 384×384 training resolution, ten denoising steps, three SDE steps in the first half, $\eta = 0.8$, guidance 1, and distinct initial noise. PickScore (Kirstain et al., 2023) supplies the training reward; advantages subtract each prompt-group mean and divide by the batch-wide reward standard deviation. LoRA rank/alpha is 32/64 on eight matched attention projections; the learner uses two updates, microbatch eight, learning rate and weight decay 10^{-4} , clip 10^{-5} , and publication every rollout. We prespecify 300 rollouts on one eight-GPU node with seed IDs 11, 22, and 33. The same budget and seed IDs apply to the pinned VeRL-Omni reference; checkpoint export and estimator parity are audited before admission. The stock pair differs in SDE strategy, step selection, and potentially normalization/anchor semantics, so it currently defines a related-recipe comparison. At least one full-budget E1 or E2 reference must pass mathematical and work parity before the paper claims reference-implementation reproduction; otherwise that claim remains unestablished. Frozen base and every 50-rollout checkpoint use a shared diffusers evaluation pipeline. The primary image endpoint is PartiPrompts HPSv3 (Ma et al., 2025); ImageReward (Xu et al., 2023) and compositional scores provide complementary checks. These are held-fixed automatic judges distinct from the optimized PickScore model, not independent human studies.

Image evaluation identity. The registry key `image/geneval2` points to UniRL’s `synthetic/test.jsonl` and a local Qwen3-VL-8B Soft-TIFA-style scorer. We name this the *synthetic compositional set*; it is not an official GenEval 2 result (Kamath et al., 2025) without a separate data-and-scorer parity audit. The prespecified image endpoints use 512×512 , 40 steps, guidance 1, and an evaluation seed of 42 shared across training seeds and checkpoints. The later E2 report describes 16 images for each of 1,632 preference prompts, rather than the original one-image preference endpoint; the exact sampler and aggregation revision must be reconciled in the released protocol. The synthetic endpoint additionally fixes its explicit linear sigma grid, prompt-hashed seeds, and 256-token text-encoder limit; the preference endpoint fixes the pinned pipeline schedule and index-based seeds. Dataset sizes, revisions, overlap counts, and scorer hashes remain open manifest fields.

The diversity guard is within-prompt mean pairwise LPIPS (Zhang et al., 2018) (AlexNet features), computed over 16 images per PartiPrompts prompt at the same evaluation setting, with identical image seeds for base and checkpoint. We average all 120 pair distances per prompt, then report the relative change in the mean across prompts and a paired prompt-bootstrap interval. A greater than 10% relative decrease is a study-specific investigation trigger; it is not a literature-established quality threshold. A near-zero base makes this relative criterion undefined and requires the absolute change to be reported. This guard diagnoses reduced diversity and is not itself a perceptual-quality metric.

Table 3: Reported E2 diffusion endpoints after 300 rollouts, with three training seeds. Changes are absolute score differences unless marked as relative percentages. Missing base/final levels are not reconstructed from differences.

Metric	Base	Rollout 300	Δ score	Δ relative
HPSv3	1.2024	9.8126	+8.6102	—
ImageReward	—	—	+1.3932	—
PickScore	—	—	+0.0970	—
Composition (VQAScore)	0.1011	0.1552	—	+53.5%
LPIPS diversity	—	—	—	-24.79%

E2 result: higher automatic scores with lower diversity. The reported mean HPSv3 gain is 8.6102, while within-prompt LPIPS changes by -24.79%. Figure 3 shows the available individual-seed differences, keeping score improvement and diversity reduction adjacent. The report records complete scoring with no errors; the preference evaluation contains 1,632 prompts and 16 images per prompt, not that many independent training runs. Synthetic composition and preference use different evaluation sets, so the preference image count must not be assigned to every metric. All three seeds exceed the prespecified diversity-investigation threshold. The reported reduction is already present at rollout 50; stopping at that checkpoint does not eliminate it, although these observations do not rule out earlier stopping or a different intervention. External automatic judges also improve, but this does not establish human preference or exclude reward exploitation. The supported conclusion is a score–diversity trade-off, not an unqualified training-validity or no-collapse result. Only endpoint summaries are available in this bundle; no intermediate learning curve is interpolated.

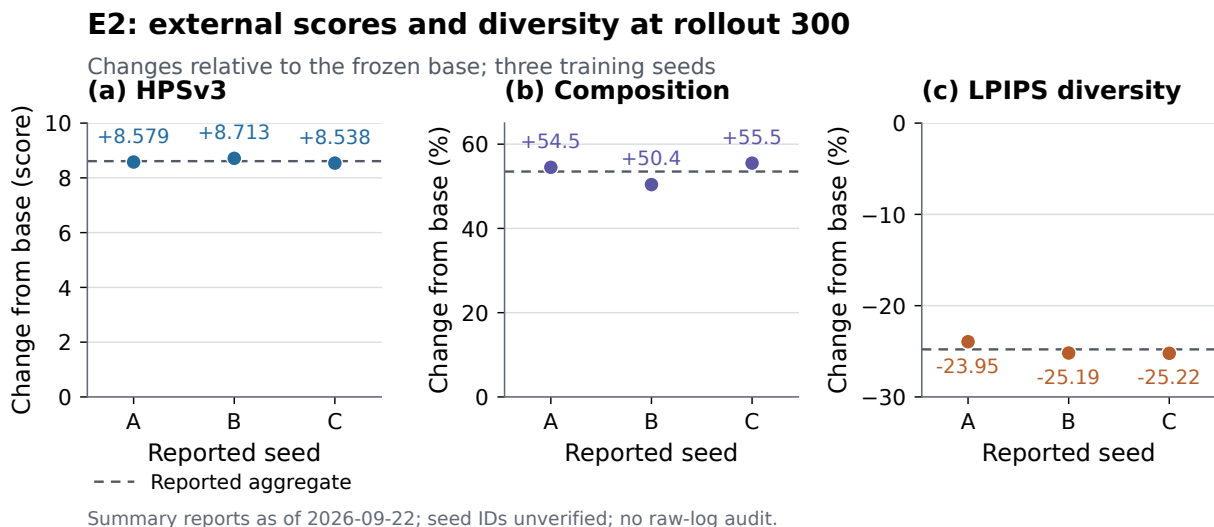


Figure 3: Reported per-seed E2 changes at the final checkpoint relative to the frozen base. Each panel retains its own metric and units; relative composition changes are not percentage-point changes. Points are training-seed summaries, not individual images. These are report-derived endpoints, with no raw-data confidence bands.

Reference comparison remains open. E2-R completed three training seeds and reports verified checkpoint exports, but no held-out head-to-head evaluation was performed. Export correctness cannot establish quality parity, and evaluation alone would not remove the native flow/CPS estimator mismatch. An explicitly labeled native-recipe comparison could provide context; strict reproduction still requires mathematical and effective-work parity.

HunyuanImage-3.0 recipe reproduction. We further evaluate UniRL on HunyuanImage-3.0-Instruct as a high-resolution text-to-image workload. The reproduced train-side recipe uses one node with eight H20 GPUs, 16 prompts and four samples per prompt, producing 64 images per rollout at 1024×1024 resolution. Sampling uses 16 inference steps and three SDE steps, and training uses FlowGRPO with PickScore reward and seed 42. The exact resolved recipe is archived under `artifacts/reported_results/2026-09-30-hunyuanimage3/`.

The reported history covers 360 rollouts, ending at a durable checkpoint. Mean training reward increased from **0.762** over the first ten rollouts to **0.870** over rollouts 351 through 360 (Figure 4). This result verifies that the HunyuanImage-3.0 recipe executes and learns in UniRL. Because the run uses one seed and has no held-out image-quality evaluation, we treat it as recipe-reproduction evidence rather than a comparative quality result.

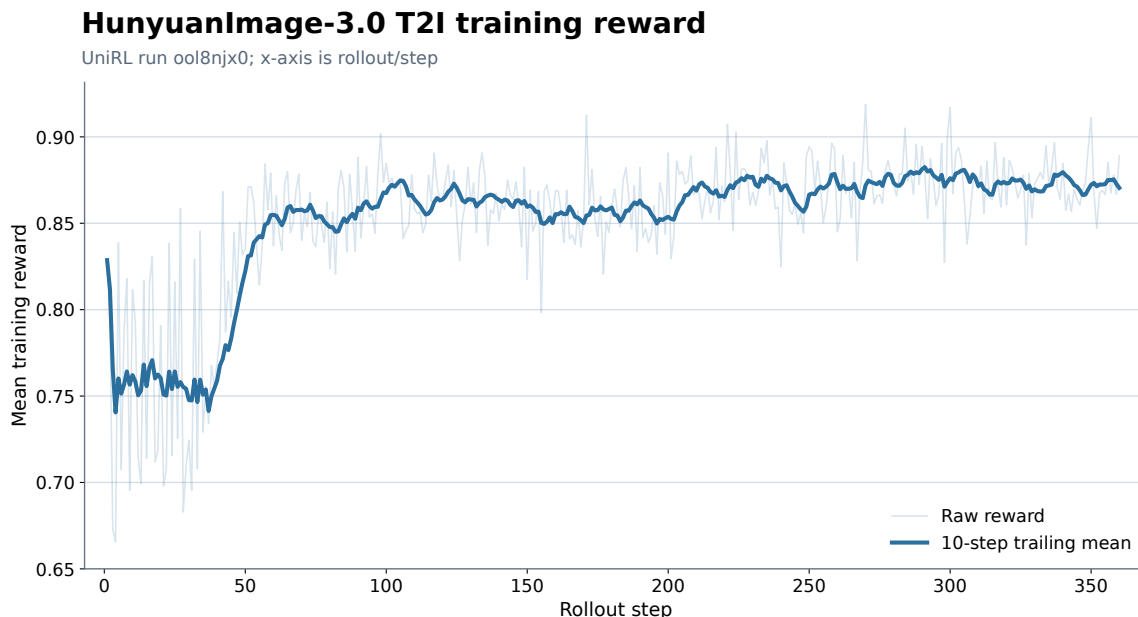


Figure 4: HunyuanImage-3.0 training reward in UniRL. The faint trace shows the reward at every rollout and the bold trace shows its trailing ten-rollout mean. The horizontal axis uses UniRL’s logical `rollout/step`, rather than W&B’s default `_step`.

MiniMax-H3 recipe reproduction. We also evaluate UniRL on MiniMax-H3 text-to-video-with-audio generation. Each logical step contains eight prompts with 16 samples per prompt, or 128 trajectories, generated at 256×384 resolution with 124 frames and ten denoising transitions. Training uses one optimizer update per logical step, learning rate 10^{-4} , ImageBind and CLAP rewards, seed 42, and one node with eight H20 GPUs. The resolved configuration is archived under `artifacts/reported_results/2026-09-30-minimax-h3/`.

The reported history ends at the durable checkpoint at logical step 200. Mean reward increases from **0.194** over the first ten steps to **0.375** over steps 191 through 200 (Figure 5). Because this is a single-seed run with no held-out video and audio quality evaluation, we use it as evidence of executable recipe and model coverage rather than as a comparative-quality, systems-performance, or time-to-quality result.

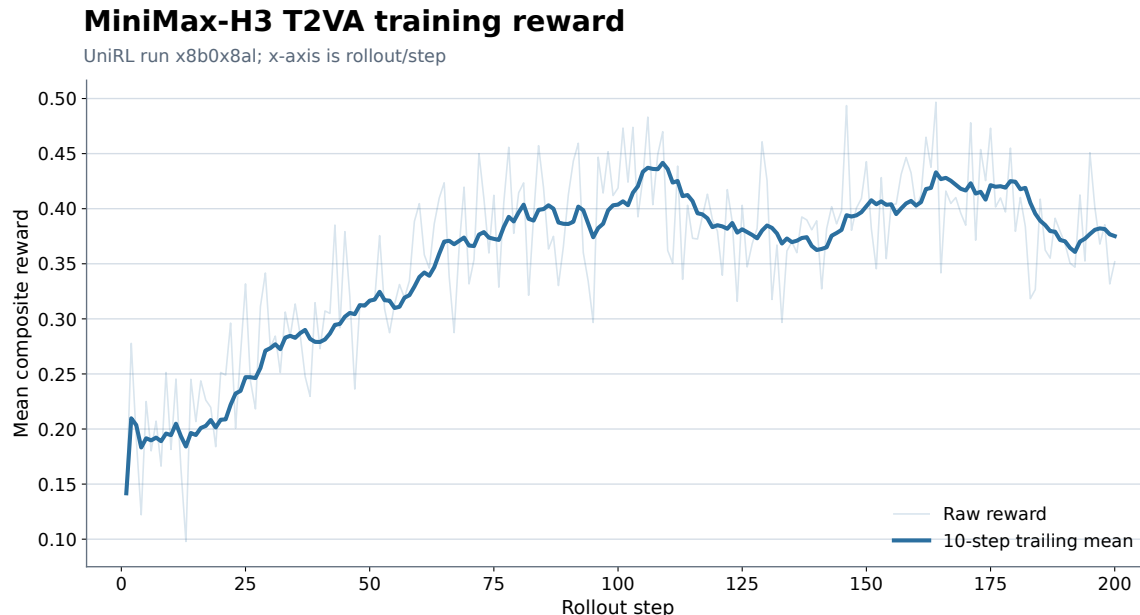


Figure 5: MiniMax-H3 training reward in UniRL. The faint trace shows the reward at every rollout and the bold trace shows its trailing ten-rollout mean. The horizontal axis uses UniRL’s logical `rollout/step`, rather than W&B’s default `_step`.

6.2 RQ2: What is end-to-end system performance?

E3 is the primary performance comparison: UniRL versus VeRL-Omni commit 01c87ee5 for the E2 SD3.5 shape-matched work unit on one reserved eight-GPU node (Huang et al., 2026). The planned work unit uses 48 prompts, 16 samples, 384×384 output, ten denoising steps, three early-window SDE steps, two optimizer updates, microbatch eight, LoRA 32/64, learning rate/weight decay 10^{-4} , clip 10^{-5} , and PickScore. The later report identifies different realized LoRA target sets, so matching rank and alpha does not establish matched trainable parameter counts. The reported protocol uses three process-level repetitions per configuration, 30 steps each, removes the first five timing observations, and retains at least 20 observations per repetition. Systems are reported to run separately on the same reserved node; cache policy and randomized execution order remain checks against the underlying run records. We show the three distinct reported configurations, without duplicating UniRL as an additional “best valid” row. The stock UniRL recipe uses `FlowSDEStrategy` and three sampled early indices; the VeRL-Omni launcher requests `cps` and an early three-step window. These may change the transition distribution and likelihood, not merely kernel implementation. Normalization, anchors, actual replay work, and reward-service GPU placement also require an audit. Until a semantically aligned pair is established, E3 is labeled a *shape-matched native-recipe comparison*; attention-backend matching alone does not make it an equivalent-algorithm framework comparison. Its speed ratio cannot be attributed solely to framework overhead.

An E1 AR comparison is secondary and remains blocked until the adapted veRL launcher matches model, data, prompt/chat template, verifier, generation budget, effective batch, estimator, token packing, optimizer work, publication, and evaluation. If semantic parity cannot be achieved, the row is omitted rather than replaced by a nearby recipe.

The full protocol requests median, mean, and p90 iteration time, throughput, phase timing, memory, and utilization; the present bundle supplies only selected aggregate timings and throughput. Strong and weak scaling use fixed global and fixed per-GPU work, respectively, after the primary single-node pair is complete. The 30-step E3 protocol does not support a time-to-quality claim; the E2-R quality comparison is still missing and E6 did not complete a threshold-based study.

Table 4: Reported E3 native-recipe systems timings and E5-T deployment timings. All rows use eight allocated GPUs. E3 and E5-T answer different questions; differences in estimators and deployment prevent a pure framework or transport attribution. Aggregate values retain the source table’s precision, not an independently established measurement accuracy.

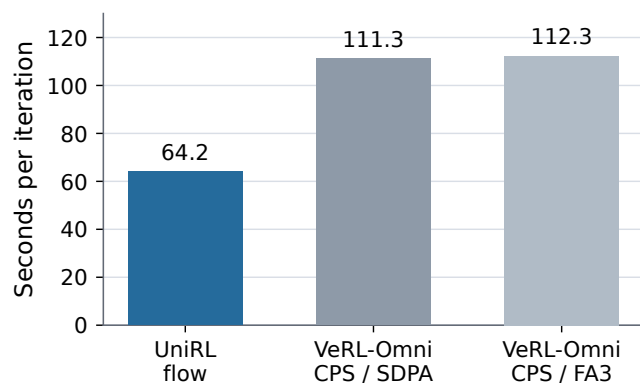
System	Configuration	s/iteration	samples/GPU-h
E3: native recipes, eight GPUs in total			
UniRL	Native flow / SDPA-class	64.2	5381
VeRL-Omni	Native CPS / SDPA	111.3	3104
VeRL-Omni	Native CPS / FA3	112.3	3079
E5-T: deployment regimes, eight GPUs in total			
UniRL	Colocated, 8 shared GPUs	64.223	–
UniRL	Separate, 4 training + 4 rollout GPUs	146.244	–

E3 result and its comparison boundary. The reported UniRL and VeRL-Omni SDPA times are 64.22 and 111.33 seconds per iteration, a ratio of approximately 1.73. The observed SDPA and FA3 times are similar; this is not an equivalence test. The result concerns the complete measured native configurations, including their estimator and LoRA-target differences. The report also records greater short-run reward improvement per step for VeRL-Omni, so iteration throughput cannot be converted into a time-to-quality advantage. Neither the report’s parameter-adjusted ratio nor its progress-normalized ratio is presented as an independently validated matched-control result. Generation phase labels are also non-equivalent: VeRL-Omni includes scoring in its generation timer, whereas UniRL separates it. We therefore omit a phase-breakdown figure until a common accounting boundary and the underlying measurements are available. The source PR body and manuscript disagree on some replicate summaries; the present figure uses only the explicitly identified aggregate table values and does not fabricate replicate points, confidence intervals, or statistical significance.

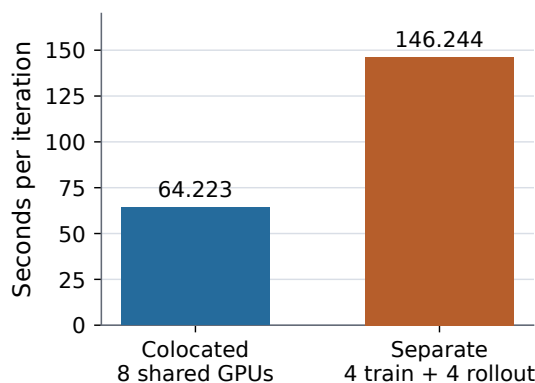
E3 and E5-T: reported iteration time

Eight GPUs in each configuration; lower is faster

(a) E3: native recipes



(b) E5-T: deployment regimes



E3 changes native recipes; E5-T changes allocation and residency together.

Reported aggregates only; no raw-log audit or uncertainty estimate.

Figure 6: Reported iteration-time summaries. Left: E3 native-recipe comparison on one eight-GPU node. Right: E5-T’s separate four-training/four-rollout allocation versus eight shared GPUs within UniRL. Lower is faster. Panels keep the external recipe comparison separate from the internal deployment comparison; no error bars are inferred from unavailable run-level data.

6.3 RQ3: What does the trajectory abstraction cost and enable?

RQ3 first tests the semantic capability that motivates the IR, then measures its cost. E4 uses the executable frozen-Qwen3-0.6B rewrite $\rightarrow$ SD3.5 path: eight original prompts, four rewrites per prompt, and eight images per rewrite, or 256 image descendants per rollout; only diffusion trains. This is a separate recipe from E2: 512×512 , ten steps, one sampled SDE index among 0–8, $\eta = 0.7$, LoRA 16/32, learning rate 3×10^{-4} , weight decay zero, clip 10^{-4} , replay anchor, two updates, and microbatch one. The primary condition computes group-relative advantages across all 32 descendants of one original prompt, while `diffusion_group_scope=rewrite` groups the eight images under each rewrite. Both conditions must serialize redacted root/part/parent IDs, shapes, reward/advantage, replay indices, group labels, and output versions. Invalid lineage remains an E0 fail-fast test rather than an intentionally corrupted quality baseline.

For image $x_{p,j,k}$, the two group keys are $g_{\text{root}} = p$ and $g_{\text{rewrite}} = (p, j)$. The intended semantic test requires an ID-joined flat-row reference to reproduce UniRL’s group memberships, advantages, and learner inputs on identical recorded rewards and replay tensors, including after root-shard round trips. The 300-rollout, three-seed training comparison changes the normalization population (32 versus eight images); any quality difference would therefore concern the grouping objective, not isolate the IR representation’s effect.

Passing that recorded-trajectory control would establish the grouping semantics on real rollout data; it would not by itself prove better task quality. The current PickScore request is conditioned by the rewritten text, so a claim about preserving the user’s original intent additionally requires lineage-keyed evaluation of every image against its root prompt. We therefore report both rewrite-conditioned training reward and frozen HPSv3/ImageReward evaluations joined to each root prompt. Evaluation generates held-out images at 512×512 , 40 steps, and guidance 1 with the base and every 50-rollout diffusion checkpoint, using one fixed manifest of four frozen rewrites per root and eight image seeds per rewrite for both conditions. Worker/adaptor RNG initialization, deterministic trainside-AR sampling, real trace dumping, and this held-out generation/evaluation path are E0 prerequisites for a multi-seed E4 result. If the two correct groupings tie, the experiment still establishes semantic expressiveness but not a quality benefit.

E4 reported results and unresolved input validity. E4a reports 24/24 successful checks in each scope and bit-exact agreement with an ID join. The fixtures are synthetic, so this is bounded semantic evidence rather than the recorded-rollout check specified above. E4b reports no consistent advantage across three seeds: the differences change sign. This does not demonstrate equivalence or establish that the effect is zero. The PR additionally discloses malformed rewrite conditioning: chat markers remain in many outputs and many rewrites exceed the CLIP token window because the intended extraction configuration was not applied. Those defects can affect both absolute quality and the grouping contrast; a statistically inconclusive outcome does not make them irrelevant. The measured configuration is retained as an exploratory result, while a corrected, separately versioned pipeline must pass a short health check before further quality comparisons.

E5-R remains unmeasured. The intended microbenchmarks separate recursive IR operations and tensor transport from model compute, varying root count, branch factor, tensor payload, token/media lengths, and part depth. Required outputs include structural-operation time, metadata bytes, dense bytes avoided at the driver, transfer time, and reassembly correctness hashes.

The broader E5 protocol compares the colocated store, per-GPU store, and TransferQueue path where hardware permits. The current report confirms that the available harness does not implement the required paired representation measurement. The missing comparison is whole-tree sharding versus explicit ID-joined flat rows on the same recorded payload, device, transport, and operations. This paired representation slice holds model work, device placement, values, grouping, and replay inputs fixed; driver RSS, bytes moved, operation time, and fraction of iteration cost measure the price of preserving structure. Semantic errors are caught in E0 and are never timed as valid work.

The minimum topology slice reuses E3’s colocated SD3.5 UniRL row and adds the same work with `sd3_vllm_omni_lora_separate.yaml`: four GPUs train and four generate while the total eight-GPU allocation remains fixed. This pair measures two deployment regimes, not a pure transport primitive—training parallelism, residency, and local versus remote LoRA publication change together and require a common phase

breakdown before attributing their costs. Broader same-/cross-GPU and cross-node backend measurements remain E5 extensions. The reported E5-T times are 64.223 seconds for shared deployment and 146.244 seconds for the separate allocation, approximately 2.277 times longer (Figure 6). The separate UniRL configuration is also slower than the colocated VerL-Omni E3 configuration. The report attributes much of the increase to reward processing, but without the phase records we retain this as a reported diagnosis, not an isolated causal result. Neither this deployment penalty nor the CPU timings below establish bounded representation overhead. We inventory which bundle/stage/adaptor code the existing E4 path uses and which trainer, scheduler, transport, and credit modules it shares. This documents reuse; reduced integration effort would require a separate held-out implementation study.

Preliminary executable evidence. We ran a source-importing regression harness against a clean UniRL checkout at f5d7104; it does not mock the trajectory classes. All ten checks passed: cross-stage lineage construction, exact tree split/concat, whole-tree select/slice, leaf-to-root reward propagation and group advantages, rejection of malformed lineage and primitive modality/batch violations, variable-length packed-text round trips, and lazy `TensorRef` selection/materialization including bounds checking. Table 5 reports a deliberately narrow timing slice from the same run. The synthetic workload has four AR children per root, one observation per AR child, two diffusion children per observation, and a four-value image payload per leaf.

Table 5: Preliminary single-process CPU trajectory-IR timings (microseconds). Python 3.12.14 and PyTorch 2.11.0; one thread; ten warm-up and 100 measured repetitions. These are regression measurements, not distributed or GPU throughput.

Roots	Leaf rows	Operation	Median	p90
8	64	<code>sample.split_concat</code>	201.3	212.0
8	64	<code>sample.select_half</code>	954.2	977.7
8	64	<code>tensorref.select_strided</code>	20.1	20.4
32	256	<code>sample.split_concat</code>	488.6	496.8
32	256	<code>sample.select_half</code>	3538.6	3615.8
32	256	<code>tensorref.select_strided</code>	74.1	76.8
128	1024	<code>sample.split_concat</code>	1653.0	1700.5
128	1024	<code>sample.select_half</code>	13893.5	14139.3
128	1024	<code>tensorref.select_strided</code>	327.9	337.5

The current `Sample.select` path forms tree-complete root shards before gathering them, so its cost grows with the full input tree rather than only the selected rows. Strided `TensorRef` selection creates many metadata spans while contiguous selection coalesces them; no dense tensor is copied until materialization. These observations motivate, but do not replace, the planned GPU transport and end-to-end ablations. The manifest, 100 per-cell raw trials, structured check outcomes, generated table rows, and SHA-256 checksums are archived under `artifacts/trajectory_ir/cpu_f5d7104_macos_arm64/`.

6.4 RQ4: Exploratory bounded-staleness observations

The prespecified study separated the allocation contrast (colocated synchronous S0 versus disaggregated D0 without overlap) from the onset of overlap (D0 versus A1), followed by publication/lag sweeps. The report states that the formal sweep was canceled on 2026-09-21; only single-seed capacity measurements and a separate quality-target pilot were completed. The capacity timings, obtained with evaluation disabled, are planning observations rather than a full-budget, equal-quality comparison and are not used for a headline speedup.

The quality-target pilot did not pass. The D0 pilot’s reported MATH-500 avg@4 decreased from 0.798 at rollout 150 to 0.773 at rollout 300, contrary to the preregistered requirement that the later checkpoint improve. No quality threshold was registered and no time-to-quality result is reported. Training reward also declined over the later window. This is a negative outcome for the tested pilot criterion; it neither identifies asynchrony as the cause nor establishes that every bounded-staleness configuration is ineffective. Two checkpoints are insufficient to locate a precise turning point.

Evaluation variability and requirements for a restart. The report’s repeated evaluation of one checkpoint found variable prompt-level avg@4 scores despite a relatively small aggregate difference. That observation is specific to the sampled checkpoint and decoding protocol; it is not a universal noise floor or a substitute for training-seed replication. A renewed study would first audit scorer behavior, request-level sampling and repeat-evaluation variability; match the synchronous/asynchronous estimator and learner settings; and record complete driver time and buffer admission, rejection, discard, retry, and completion accounting. The source-audited `perf/step_time_s` alone omits outer-loop publication, evaluation, and checkpoint work. Only after establishing a suitable learning budget and a separately piloted target should a replicated formal grid support a lag–quality–time conclusion. The original grid and preflight controls remain documented in `GPU_HANDOFF_README.md`; they are not represented here as completed experiments.

6.5 Fairness and statistical protocol

Effective work is matched before wall-clock time: prompts, samples per prompt, decoded resolution or maximum tokens, rollout steps, replayed steps/tokens, microbatching semantics, and optimizer updates. Primary full-system metrics count all allocated training, rollout, and reward resources on both sides. Reward placement and concurrency must be disclosed; a reward-excluded number is supplementary and carries a separate service-cost report. We prespecify warmup removal, use at least 20 steady-state iterations for throughput summaries, and report the full distribution rather than a single minimum. Training summaries use the seed as the statistical unit. The present figures show the available individual-seed summaries; a full release should additionally report justified seed-level uncertainty. Bootstrap intervals over prompts or images must be labeled separately and cannot replace seed uncertainty. The complete systems release must show all process repetitions and the post-warmup per-step distribution; uncertainty of a speed ratio must use repetitions, not correlated steps, as its independent units. The current report-derived summaries do not satisfy that full distribution-release requirement. The final checkpoint and named primary endpoint are fixed in advance; the auxiliary metrics diagnose failure modes and are not alternative endpoints chosen after seeing results. Any future E6 time-to-quality study must use MATH-500 and a target registered from a separate pilot before the formal grid, bracket first observed crossings by evaluation times, and censor runs that never cross. The canceled study produced no such result. Elapsed time spans initialization, training, publication, checkpointing, and evaluation; steady-state throughput is reported separately. All published cells require complete scoring or an explicit missingness analysis; silently averaging only successful evaluator responses is inadmissible.

7 Related Work

LLM post-training systems. HybridFlow/veRL combines single- and multi-controller execution and introduces hierarchical APIs for mapping RLHF dataflows to devices (Sheng et al., 2024). OpenRLHF uses Ray, vLLM, and DeepSpeed to schedule the models in an RLHF workflow (Hu et al., 2024). ROLL emphasizes a single-controller abstraction, parallel workers, sample lifecycle scheduling, and flexible device mapping (Wang et al., 2025). NeMo-RL provides scalable training and generation backends and multimodal model support (NVIDIA Corporation, 2025). UniRL shares their concern for composable orchestration; its intended distinction is the lineage- and modality-aware trajectory boundary across AR, diffusion, and cross-stage generation. That distinction must be established by RQ3 rather than a feature table.

Asynchronous post-training. AReaL fully decouples generation and training and controls staleness to improve language-reasoning throughput (Fu et al., 2025). Relax uses service-level separation, a TransferQueue data bus, and configurable staleness for omni-modal post-training (Zhang et al., 2026). slime supports persistent asynchronous rollout generation around a training/rollout/data-buffer loop (Zhu et al., 2025). These systems make asynchrony an execution design space, not a UniRL novelty claim. UniRL implements a version-stamped group and publication-boundary discipline shared by AR and diffusion paths; RQ4 tests whether that realization is useful, without claiming a new asynchronous RL algorithm.

Multimodal and diffusion post-training systems. VeRL-Omni extends veRL toward diffusion, unified, and omni-modality models and integrates vLLM-Omni rollout (Huang et al., 2026). Relax is explicitly

omni-native (Zhang et al., 2026). Accordingly, “supporting image, video, or audio” cannot be the paper’s contribution. The comparison instead asks how ancestry, typed replay state, and movement compose at the trajectory boundary. E4 tests one concrete workflow; a universal claim of easier integration is outside its evidence scope.

Diffusion RL algorithms. DDPO casts denoising as a multi-step decision process (Black et al., 2023). Flow-GRPO converts flow sampling to an SDE and applies group-relative optimization (Liu et al., 2025); DanceGRPO generalizes related ideas across visual generation tasks (Xue et al., 2025); DiffusionNFT instead optimizes a forward-process objective from reward-ranked generations (Zheng et al., 2025). UniRL implements several algorithm families but does not claim their mathematical ideas. Its algorithm interface keeps their incompatible replay state explicit while reusing scheduling and training mechanics.

8 Limitations and Open Evidence

Integration is not automatic. Each new architecture still needs a bundle, stages, conditions, and adapters for the selected rollout runtime. A common trajectory type does not eliminate tokenizer, scheduler, latent-layout, LoRA-target, or serving-version incompatibilities. The current ordered-frontier operations also assume compatible depths and parent-contiguous groups; arbitrary ragged forests, shared-field equality, and adversarial row order are not uniformly validated by the container itself.

Topology is configured, not optimized. Placement scopes make resource choices explicit but do not search for the best split. The best topology depends on model memory, reward cost, rollout variance, interconnect, and publication size.

Asynchronous execution is bounded and partially specialized. Publication is a barrier, and current AR and diffusion paths impose different admissible in-flight structure. Fault isolation is limited by the shared Ray runtime and role lifecycle; the system does not yet claim service-level recovery comparable to dedicated service architectures.

The empirical case is incomplete. The reported GPU summaries support qualified observations about diffusion scores and diversity, native-recipe iteration time, and deployment sensitivity. They do not establish mathematically aligned reference reproduction, primary-endpoint AR improvement, a quality advantage from lineage grouping, bounded IR overhead, scaling, or a time-to-quality benefit from asynchrony. The raw GPU records were unavailable during this integration, and the release still needs exact run manifests, consistent aggregate provenance, and independently regenerated results. The malformed rewrite pipeline and synthetic-only E4a fixtures further limit the evidence for the central cross-stage claim.

9 Conclusion

UniRL treats post-training as a trajectory program whose semantic object survives changes in generative family and execution topology. Lineage, typed modality values, replay segments, reward, credit, and policy version live in one IR; rollout, reward, training, transport, and publication are logical roles mapped separately to resources. This design is most valuable if it passes two empirical tests: credible learning behavior across representative AR and diffusion workloads, and competitive end-to-end execution without weakening semantic parity. Those tests are necessary but not sufficient for the paper’s trajectory-centric thesis: a real composed run must also demonstrate lineage-defined credit/grouping, and the measured IR/topology costs must bound the price of preserving that structure. The current reported evidence shows diffusion score improvements accompanied by reduced diversity, faster iteration of one native configuration, and a substantial penalty for the measured separate deployment. The full trajectory-centric claim remains open until real-rollout lineage validation and matched representation-cost measurements complement those observations. Bounded staleness remains an exploratory operating-region question, with no demonstrated time-to-quality benefit in the canceled study.

Reproducibility statement

The source characterization in this draft refers to UniRL commit `f5d710406b215bb7a0b387fdd37e4d4778b92338` (2026-08-20). The manuscript repository records the claim-to-evidence plan in `PAPER_PLAN.md`. The cluster-ready execution protocol, configuration-parity gates, metric inventory, and immutable run-directory contract are specified in `EXPERIMENT_RUNBOOK.md`; the command-oriented transition to a fresh GPU-side agent is in `GPU_HANDOFF_README.md`. Both documents intentionally contain no results. The current result audit is in `EXPERIMENT_RESULTS.md`. The dated bundle `artifacts/reported_results/2026-09-22/` stores compact source snapshots, source-level caveats, reported numerical inputs, and generated tables and figures. These reproduce the transcription and presentation of the reports, not the original GPU measurements. The later bundles `artifacts/reported_results/2026-09-30-hunyuanimage3/` and `artifacts/reported_results/2026-09-30-minimax-h3/` store the resolved recipes, direct W&B history exports, and rendered learning curves. All graphics are committed figure assets included by relative paths, so Overleaf does not require Python or shell escape to build the manuscript. The preliminary RQ3 evidence is regenerated by `experiments/trajectory_ir/run_cpu_evidence.py`; its manifest pins the source state, command, environment, workload, warm-up, repetitions, and timer. **TODO[data]: Before release, attach raw GPU artifacts and run identities, reconcile the E3 replicate summaries and E2 evaluation protocol, complete the primary AR and core IR evidence, and add author metadata and a durable artifact URL.**

References

- Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. 2023. URL <https://arxiv.org/abs/2305.13301>.
- Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. 2025. URL <https://arxiv.org/abs/2505.24298>.
- Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. 2024. URL <https://arxiv.org/abs/2405.11143>.
- Yongxiang Huang, Cheung Kawai, Jingan Zhou, Yingshu Chen, openYuanrong Team, and Xibin Wu. VerL-Omni: Easy, fast, and stable RL training for diffusion and omni-modality models. GitHub repository, 2026. URL <https://github.com/verl-project/verl-omni>.
- Amita Kamath, Kai-Wei Chang, Ranjay Krishna, Luke Zettlemoyer, Yushi Hu, and Marjan Ghazvininejad. GenEval 2: Addressing benchmark drift in text-to-image evaluation. 2025. URL <https://arxiv.org/abs/2512.16853>.
- Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. 2023. URL <https://arxiv.org/abs/2305.01569>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online RL. 2025. URL <https://arxiv.org/abs/2505.05470>.
- Yuhang Ma, Yunhao Shui, Xiaoshi Wu, Keqiang Sun, and Hongsheng Li. HPSv3: Towards wide-spectrum human preference score. 2025. URL <https://arxiv.org/abs/2508.03789>.
- Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for

-
- emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation*, 2018. URL <https://www.usenix.org/conference/osdi18/presentation/moritz>.
- NVIDIA Corporation. NeMo RL: A scalable and efficient post-training library. GitHub repository, 2025. URL <https://github.com/NVIDIA-NeMo/RL>.
- Ruoyu Qin et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. 2024. URL <https://arxiv.org/abs/2407.00079>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017. URL <https://arxiv.org/abs/1707.06347>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. 2024. URL <https://arxiv.org/abs/2402.03300>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. 2024. URL <https://arxiv.org/abs/2409.19256>.
- Stability AI. Stable diffusion 3.5 medium model card. Hugging Face model repository, 2024. URL <https://huggingface.co/stabilityai/stable-diffusion-3.5-medium>.
- Weixun Wang et al. ROLL: An efficient and user-friendly scaling library for reinforcement learning with large language models. 2025. URL <https://arxiv.org/abs/2506.06122>.
- Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and evaluating human preferences for text-to-image generation. 2023. URL <https://arxiv.org/abs/2304.05977>.
- Zeyue Xue, Jie Wu, Yu Gao, Fangyuan Kong, Lingting Zhu, Mengzhao Chen, Zhiheng Liu, Wei Liu, Qiushan Guo, Weilin Huang, and Ping Luo. DanceGRPO: Unleashing GRPO on visual generation. 2025. URL <https://arxiv.org/abs/2505.07818>.
- An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. 2025. URL <https://arxiv.org/abs/2505.09388>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. 2025. URL <https://arxiv.org/abs/2503.14476>.
- Liujie Zhang, Benzhe Ning, Rui Yang, Xiaoyan Yu, Jiaxing Li, Lumeng Wu, Jia Liu, Minghao Li, Weihang Chen, Weiqi Hu, and Lei Zhang. Relax: An asynchronous reinforcement learning engine for omni-modal post-training at scale. 2026. URL <https://arxiv.org/abs/2604.11554>.
- Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. URL <https://arxiv.org/abs/1801.03924>.
- Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Bali, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. 2023. URL <https://arxiv.org/abs/2304.11277>.
- Kaiwen Zheng, Huayu Chen, Haotian Ye, Haoxiang Wang, Qinsheng Zhang, Kai Jiang, Hang Su, Stefano Ermon, Jun Zhu, and Ming-Yu Liu. DiffusionNFT: Online diffusion reinforcement with forward process. 2025. URL <https://arxiv.org/abs/2509.16117>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, and Joseph E. Gonzalez. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2312.07104>.

Zilin Zhu, Chengxing Xie, Xin Lv, and contributors. slime: An LLM post-training framework for RL scaling. GitHub repository, 2025. URL <https://github.com/THUDM/slime>.

A Claim-to-source audit

Table 6 records the executable source used to support mechanism descriptions. It does not prove performance or successful execution for every configuration.

Table 6: Primary code evidence at commit f5d7104.

Mechanism	Primary source symbols
Trajectory IR	<code>unirl/types/sample.py</code> : <code>Part</code> , <code>Sample</code> ; <code>unirl/types/primitives.py</code> ; <code>unirl/types/segments/</code>
Batch algebra	<code>unirl/distributed/tensor/batch.py</code> : <code>FieldKind</code> , <code>Batch</code>
Tensor references	<code>unirl/distributed/tensor/ref.py</code> : <code>TensorSpan</code> , <code>TensorRef</code> ; <code>transport.py</code> : <code>TensorTransport</code>
Roles and placement	<code>unirl/distributed/group/device_pool.py</code> , <code>placement.py</code> , <code>handle.py</code> , <code>dispatch.py</code>
Rollout boundary	<code>unirl/rollout/engine/base.py</code> : <code>BaseRolloutEngine</code> ; concrete engines under <code>unirl/rollout/engine/</code>
Training boundary	<code>unirl/algorithms/base.py</code> : <code>StageAlgorithm</code> ; <code>unirl/train/stack/base.py</code> : <code>TrainStack</code>
Sync trainers	<code>unirl/trainer/ar.py</code> : <code>ARTrainer</code> ; <code>unirl/trainer/diffusion.py</code> : <code>DiffusionTrainer</code>
Async scheduler	<code>unirl/trainer/async_rollout.py</code> ; <code>unirl/rollout/manager/</code>
Weight publication	<code>unirl/distributed/weight_sync/</code> ; engine-specific receivers under <code>unirl/rollout/engine/</code>
Instrumentation	<code>unirl/utils/profiling.py</code> , <code>memory_monitor.py</code> , <code>wandb_logger.py</code>

B Required run artifact schema

Every result row must resolve to a run directory containing:

- source and baseline commit hashes, submodule hashes, and dirty patches;
- the launch command and fully resolved configuration;
- model, reward, dataset, and tokenizer revisions or content hashes;
- GPU model/count, node/interconnect description, driver, CUDA, PyTorch, generation-engine, and training-backend versions;
- raw stdout/stderr, per-step structured timing, memory and utilization traces, checkpoint metadata, and evaluation outputs;
- seed, warmup exclusion rule, failure/retry accounting, and statistical aggregation script.

For training runs, serialized trace samples must redact private data while retaining lineage IDs, shapes, dtypes, policy versions, reward components, and segment index metadata. For system baselines, an alignment manifest must explain every nonidentical setting and whether it changes effective work.

C Correctness status and remaining checks

Executed locally. The preliminary RQ3 harness performs ten fail-fast checks on the audited source: lineage-valid cross-stage construction; exact split/concat; tree-complete select and slice; mean reward propagation plus group-normalized advantages; rejection of bad lineage, modality keys, and primitive batch sizes; packed variable-length text select/concat; and lazy tensor-reference selection, materialization, identity, and bounds behavior. The structured outcomes and data-sensitive SHA-256 round-trip hash are stored with the run artifacts.

Required before complete evidence admission. The subsequent reports describe E0 correctness closure, but its underlying test and configuration artifacts were not revalidated in this integration. The following system-level checks are the evidence requirements, not a claim that every experiment still lacks each check:

1. **Rollout-replay parity:** before the first update, compare engine-emitted and train-side replay log probabilities or transition statistics under stated numerical tolerances.
2. **Publication parity:** repeat the parity check after each LoRA or full-weight publication path used in an experiment.
3. **Batch-version integrity:** verify that an optimizer batch never mixes behavior versions when the configured algorithm assumes one version.
4. **Boundary integrity:** checkpoint and evaluation begin only after the async manager is empty; resume restores the optimizer version and republishes it before new rollout.
5. **Work accounting:** compare generated tokens or pixels, denoising and replay steps, scored samples, and committed optimizer updates across systems.
6. **Structural preconditions:** validate unique root IDs, equal frontier depth, parent-major grouping, shared-field equality, and version provenance before merging or propagating credit; check against an ID-joined reference.